

Introduction

Interval
Arithmetic

Interval Newton
Method

Krawczyk
Method

Algorithm

Conclusion

Automated Root Certification in Lean for Transversal Polynomial Systems

Dattabhasvant Ghadiyaram

Indian Institute of Science, Bengaluru

Motivation

Introduction

Interval
Arithmetic

Interval Newton
Method

Krawczyk
Method

Algorithm

Conclusion

$$G = \langle x, y | xyx = yxy \rangle$$

Motivation

Introduction

Interval
Arithmetic

Interval Newton
Method

Krawczyk
Method

Algorithm

Conclusion

$$G = \langle x, y \mid xyx = yxy \rangle$$

Consider $\phi(G) \rightarrow SU(2)$, where $\phi : x \mapsto X$ and $\phi : y \mapsto Y$ where

$$X = \begin{pmatrix} x_1 + ix_2 & 0 \\ 0 & x_1 - ix_2 \end{pmatrix} \quad Y = \begin{pmatrix} y_1 + iy_2 & y_3 \\ y_3 & y_1 - iy_2 \end{pmatrix}$$

To check if this homomorphism is non-trivial, solve the system of 5 unknowns and 5 polynomial equations:

$$\det X = 1$$

$$\det Y = 1$$

$$XYX = YXY$$

Motivation

Introduction

Interval
Arithmetic

Interval Newton
Method

Krawczyk
Method

Algorithm

Conclusion

Show what the 5 eqns are... Maybe

Gröbner Basis and CAD

Introduction

Interval
Arithmetic

Interval Newton
Method

Krawczyk
Method

Algorithm

Conclusion

Take few slides, talk about scalability issue for Grobner Basis

Interval Arithmetic

Introduction

Interval
Arithmetic

Interval Newton
Method

Krawczyk
Method

Algorithm

Conclusion

Floating Point Arithmetic

- $\pi \rightarrow 3.14159$
- When we evaluate some f , we only know $f(3.14159)$
- For proofs, separately need to reason $|f(\pi) - f(3.14159)| < \epsilon$
because $|\pi - 3.14159| < \delta$

Interval Arithmetic

Introduction

Interval
Arithmetic

Interval Newton
Method

Krawczyk
Method

Algorithm

Conclusion

Floating Point Arithmetic

- $\pi \rightarrow 3.14159$
- When we evaluate some f , we only know $f(3.14159)$
- For proofs, separately need to reason $|f(\pi) - f(3.14159)| < \epsilon$
because $|\pi - 3.14159| < \delta$

Interval Arithmetic

- $\pi \rightarrow [3.1415, 3.1416]$
- If f can be applied to $[3.1415, 3.1416]$ to get $[a, b]$
- We want $\forall x \in [3.1415, 3.1416], f(x) \in [a, b]$

Dyadic Intervals

- $dy[[a, b]] = \{x \in \mathbb{R} \mid a \leq x \leq b\}$ where a, b are Dyadic, i.e. $p/2^k$ for an odd $p \in \mathbb{N}$ and $k \in \mathbb{Z}$

```
/- Dyadic Intervals are defined as a pair  
of Dyadic along with a proof that left ≤ right -/  
structure DyadicInterval where  
  left : Dyadic  
  right : Dyadic  
  isValid : left ≤ right  
  deriving DecidableEq
```

- Width : $b - a$
- Midpoint : $(a + b)/2$
- Absolute Value : $\max(|a|, |b|)$
- Subset : $a_1 \leq a_2$ and $b_2 \leq b_1$
- Intersection : $dy[[\max(a_1, a_2), \min(b_1, b_2)]]$ or `none`

Arithmetic Operations

- **Addition** : $dy[[a,b]] + dy[[c,d]] = dy[[a+c, b+d]]$
- **Subtraction** : $dy[[a,b]] - dy[[c,d]] = dy[[a-d, b-c]]$

Introduction

Interval
Arithmetic

Interval Newton
Method

Krawczyk
Method

Algorithm

Conclusion

Arithmetic Operations

- **Addition** : $\text{dy}[[a,b]] + \text{dy}[[c,d]] = \text{dy}[[a+c, b+d]]$
- **Subtraction** : $\text{dy}[[a,b]] - \text{dy}[[c,d]] = \text{dy}[[a-d, b-c]]$
- **Multiplication** : $\text{dy}[[a,b]] * \text{dy}[[c,d]] = \text{dy}[[\min(ac, ad, bc, bd), \max(ac, ad, bc, bd)]]$
- **Division (Approximate)** : If $0 \notin [[c, d]]$,
$$\frac{[[a,b]]}{[[c,d]]} = [[\min(a/c, a/d, b/c, b/d), \max(a/c, a/d, b/c, b/d)]]$$

(Rounded outwards to nearest Dyadic of specified precision)

Introduction

Interval
Arithmetic

Interval Newton
Method

Krawczyk
Method

Algorithm

Conclusion

Arithmetic Operations

Introduction

Interval
Arithmetic

Interval Newton
Method

Krawczyk
Method

Algorithm

Conclusion

- **Addition** : $\text{dy}[[a,b]] + \text{dy}[[c,d]] = \text{dy}[[a+c, b+d]]$
- **Subtraction** : $\text{dy}[[a,b]] - \text{dy}[[c,d]] = \text{dy}[[a-d, b-c]]$
- **Multiplication** : $\text{dy}[[a,b]] * \text{dy}[[c,d]] = \text{dy}[[\min(ac, ad, bc, bd), \max(ac, ad, bc, bd)]]$
- **Division (Approximate)** : If $0 \notin [[c, d]]$,
 $\frac{[[a,b]]}{[[c,d]]} = [[\min(a/c, a/d, b/c, b/d), \max(a/c, a/d, b/c, b/d)]]$
(Rounded outwards to nearest Dyadic of specified precision)
- **Exponentiation**
 - $\text{dy}[[a,b]]^{2k+1} = \text{dy}[[a^{2k+1}, b^{2k+1}]]$
 - $\text{dy}[[-a, b]]^{2k} = \text{dy}[[0, \max((-a)^{2k}, b^{2k})]]$
 - $\text{dy}[[a, b]]^{2k} = \text{dy}[[\min(a^{2k}, b^{2k}), \max(a^{2k}, b^{2k})]]$

Properties

Introduction

Interval
Arithmetic

Interval Newton
Method

Krawczyk
Method

Algorithm

Conclusion

The arithmetic operations are correct and optimal, except for division. They obey the following:

- **Soundness** : $\forall x \in I, y \in J, x \circ y \in I \circ J$
- **Sharpness** : $\forall z \in I \circ J, \exists x \in I, y \in J$ such that $x \circ y = z$

Division is sound, but not sharp.

Dependency Problem

- No additive inverse

$$dy[[-1, 1]] - dy[[-1, 1]] = dy[[-1 - 1, 1 - (-1)]] = dy[[-2, 2]]$$

Introduction

**Interval
Arithmetic**

Interval Newton
Method

Krawczyk
Method

Algorithm

Conclusion

Dependency Problem

- No additive inverse

$$dy[[-1, 1]] - dy[[-1, 1]] = dy[[-1 - 1, 1 - (-1)]] = dy[[-2, 2]]$$

- Distributivity fails

$$dy[[-1, 2]] * (dy[[1, 2]] + dy[[-2, -1]]) = dy[[-2, 2]]$$

$$dy[[-1, 2]] * dy[[1, 2]] + dy[[-1, 2]] * dy[[-2, -1]] = dy[[-6, 6]]$$

Introduction

Interval
Arithmetic

Interval Newton
Method

Krawczyk
Method

Algorithm

Conclusion

Dependency Problem

- No additive inverse

$$dy[[-1, 1]] - dy[[-1, 1]] = dy[[-1 - 1, 1 - (-1)]] = dy[[-2, 2]]$$

- Distributivity fails

$$dy[[-1, 2]] * (dy[[1, 2]] + dy[[-2, -1]]) = dy[[-2, 2]]$$

$$dy[[-1, 2]] * dy[[1, 2]] + dy[[-1, 2]] * dy[[-2, -1]] = dy[[-6, 6]]$$

- Overestimation when evaluating polynomials. Consider $f(x) = x^2 - x$ over $dy[[0, 1]]$

$$\text{True Range : } dy[[-0.25, 0]] \quad \text{Computed Range : } dy[[-1, 1]]$$

Interval Vectors and Matrices

Interval Vectors

- An interval vector $\mathbf{X} = (X_1, X_2, \dots, X_n)$ is an ordered tuple of intervals, representing an interval box - Convex, compact set in $(\mathbb{R}^n, d_\infty)$
- Implemented using the dependent type in Lean, `Vector DyadicInterval n` - Array along with a proof of length
- Isomorphic to the Pi-type, `Fin n → DyadicInterval`

Introduction

Interval
Arithmetic

Interval Newton
Method

Krawczyk
Method

Algorithm

Conclusion

Interval Vectors and Matrices

Interval Vectors

- An interval vector $\mathbf{X} = (X_1, X_2, \dots, X_n)$ is an ordered tuple of intervals, representing an interval box - Convex, compact set in $(\mathbb{R}^n, d_\infty)$
- Implemented using the dependent type in Lean, `Vector DyadicInterval n` - Array along with a proof of length
- Isomorphic to the Pi-type, `Fin n → DyadicInterval`

Interval Matrices

- Implemented as a `Vector` of interval vectors. Isomorphic to Mathlib's `Matrix` type
- The norm $\|\cdot\|_\infty$ is the row-sum norm, induced by d_∞

Introduction

Interval
Arithmetic

Interval Newton
Method

Krawczyk
Method

Algorithm

Conclusion

Interval Extensions

Introduction

Interval
Arithmetic

Interval Newton
Method

Krawczyk
Method

Algorithm

Conclusion

- For a function $f : \mathbb{R}^n \rightarrow \mathbb{R}^m$, an **Interval Extension** is any interval valued function F , such that $\forall \mathbf{x} \in \mathbb{R}^n, F(\mathbf{x}) = f(\mathbf{x})$
- Inclusion Isotonicity : $\mathbf{X} \subseteq \mathbf{Y} \Rightarrow F(\mathbf{X}) \subseteq F(\mathbf{Y})$

Interval Extensions

Introduction

Interval
Arithmetic

Interval Newton
Method

Krawczyk
Method

Algorithm

Conclusion

- For a function $f : \mathbb{R}^n \rightarrow \mathbb{R}^m$, an **Interval Extension** is any interval valued function F , such that $\forall \mathbf{x} \in \mathbb{R}^n, F(\mathbf{x}) = f(\mathbf{x})$
- Inclusion Isotonicity : $\mathbf{X} \subseteq \mathbf{Y} \Rightarrow F(\mathbf{X}) \subseteq F(\mathbf{Y})$
- For well-behaved functions, **Mean Value Form**:

$$F_{mv}(\mathbf{X}) = f(\mathbf{x}_m) + \sum_{i=1}^n D_i F(\mathbf{X}) \cdot (X_i - x_{m,i})$$

- By the Mean Value theorem for f , $\forall \mathbf{x} \in X, f(\mathbf{x}) \in F_{mv}(X)$

Computable Polynomials

Introduction

Interval
Arithmetic

Interval Newton
Method

Krawczyk
Method

Algorithm

Conclusion

- $p : \text{MvRatPol } n$ represents $p \in \mathbb{Q}[X_1, \dots, X_n]$; strictly a list of $\mathbb{Q} \times \mathbb{N}^n$ tuples
- Can be evaluated on $\mathbf{x} \in \mathbb{R}^n$ for proofs and on $\mathbf{X}$, an interval vector for computation
- Symbolic derivative $\text{MvRatPol } n \rightarrow \text{MvRatPol } n$
- $\text{System } m \ n$ represents a system with m polynomial equations in n unknown; precisely $f : \mathbb{R}^n \rightarrow \mathbb{R}^m$ where each component is a polynomial described above

Interval Newton Method

Introduction

Interval
Arithmetic

Interval Newton
Method

Krawczyk
Method

Algorithm

Conclusion

- For an interval vector $\mathbf{X}$ and a system S , if its Jacobian, $J_S(\mathbf{X})$ is regular ($0 \notin \det J_S(\mathbf{X})$), we define the Newton Operator;

$$N(\mathbf{X}) = \mathbf{x}_m - J_S(\mathbf{X})^{-1} \cdot S(\mathbf{x}_m)$$

- Implemented for polynomials in one variable;

$$N(I) = I_m - \frac{f(I_m)}{F'(I)} \quad \text{if } 0 \notin F'(I)$$

- $N(\mathbf{X}) \subseteq \mathbf{X} \Rightarrow \mathbf{X}$ contains exactly one root of S
- $N(\mathbf{X})$ and $\mathbf{X}$ Disjoint $\Rightarrow \mathbf{X}$ has no root of S .

Krawczyk Method

- For a pre-conditioner matrix Y , define the interval-vector valued Krawczyk operator;

$$K(\mathbf{X}) = \mathbf{x}_m - Y \cdot S(\mathbf{x}_m) + (I - Y * J_S(\mathbf{X})) \cdot (\mathbf{X} - \mathbf{x}_m)$$

- Y is chosen as inverse of the midpoint-matrix of $J_S(X)$

Existence & Uniqueness

If $\|I - YJ_S(\mathbf{X})\| < 1$ and $\det(Y) \neq 0$

$K(\mathbf{X}) \subseteq \mathbf{X} \Rightarrow \mathbf{X}$ contains exactly one root of S

Exclusion

$K(\mathbf{X}) \cap \mathbf{X} = \emptyset \Rightarrow \mathbf{X}$ has no root of S

Proof of Correctness

Define the pointwise Krawczyk function $k : \mathbb{R}^n \rightarrow \mathbb{R}^n$ given by $k(\mathbf{x}) = \mathbf{x} - Y \cdot S(\mathbf{x})$.

Soundness

$\forall \mathbf{x} \in \mathbf{X}, k(\mathbf{x}) \in K(\mathbf{X})$, equivalently $k(\mathbf{X}) \subseteq K(\mathbf{X})$

Introduction

Interval
Arithmetic

Interval Newton
Method

Krawczyk
Method

Algorithm

Conclusion

Proof of Correctness

Define the pointwise Krawczyk function $k : \mathbb{R}^n \rightarrow \mathbb{R}^n$ given by $k(\mathbf{x}) = \mathbf{x} - Y \cdot S(\mathbf{x})$.

Soundness

$\forall \mathbf{x} \in \mathbf{X}, k(\mathbf{x}) \in K(\mathbf{X})$, equivalently $k(\mathbf{X}) \subseteq K(\mathbf{X})$

- Apply Mean Value Theorem for each component S_i :
 $\exists \xi_i \in \mathbf{X}$ such that $S_i(\mathbf{x}) = S_i(\mathbf{x}_m) - J_{S,i}(\xi_i)(\mathbf{x} - \mathbf{x}_m)$
where $J_{S,i}$ is the i^{th} row of J_S .
- Define the matrix $\mathcal{J}$ with rows $J_{S,i}(\xi_i)$.
Since $J_{S,i}(\xi_i) \in J_{S,i}(\mathbf{X})$, $\mathcal{J} \in J_S(\mathbf{X})$.
- Then, $k(\mathbf{x})$ can be expanded:

$$\begin{aligned} k(\mathbf{x}) &= \mathbf{x} - Y \cdot (S(\mathbf{x}_m) - \mathcal{J} \cdot (\mathbf{x} - \mathbf{x}_m)) \\ &= \mathbf{x}_m - Y \cdot S(\mathbf{x}_m) + (I - Y\mathcal{J}) \cdot (\mathbf{x} - \mathbf{x}_m) \end{aligned}$$

Proof of Correctness (contd.)

- Hypotheses

$$K(\mathbf{X}) \subseteq \mathbf{X} \quad \|I - YJ_S(\mathbf{X})\| < 1 \quad \det(Y) \neq 0$$

- $\|k'(\mathbf{x})\| = \|I - YJ_S(\mathbf{x})\| \leq \|I - YJ_S(\mathbf{X})\| < 1$
 $\Rightarrow k$ is a contracting map on $\mathbf{X}$
- BFPT $\Rightarrow \exists! \mathbf{x} \in X$ such that $k(\mathbf{x}) = \mathbf{x} \Rightarrow Y \cdot S(\mathbf{x}) = 0$
- $\det(Y) \neq 0 \Rightarrow S(\mathbf{x}) = 0$
- If $\mathbf{X}$ contains $\mathbf{x}$ such that $S(\mathbf{x}) = 0$, then $k(\mathbf{x}) = \mathbf{x}$ which implies it must be in $K(\mathbf{X})$
- Then, if $K(\mathbf{X})$ and $\mathbf{X}$ are disjoint, there are no roots

Algorithm

Algorithm 1 IsolateRoots

- 1: **Input** : $S \leftarrow \text{System } n \times n$, $X \leftarrow \text{Vecterval } n$, maxDepth
 - 2: **Output** : (C, U)
 - 3: **if** $0 \notin S(X)$ **then return** $(\emptyset, \emptyset)$ ▷ Pruned
 - 4: **if** $\det(Y) \neq 0$ and $\|I - YJ_S(X)\| < 1$ **then**
 - 5: Compute $K(X)$
 - 6: **if** $X \cap K(X) = \emptyset$ **then return** $(\emptyset, \emptyset)$ ▷ Pruned
 - 7: **if** $K(X) \subseteq X$ **then return** $(\{X\}, \emptyset)$ ▷ Certified
 - 8: **end if**
 - 9: **if** $\text{maxDepth} = 0$ **then return** $(\emptyset, \{X\})$ ▷ Unresolved
 - 10: $(X_1, X_2) \leftarrow \text{Split}(X)$ ▷ Bisect
 - 11: $(C_1, U_1) \leftarrow \text{IsolateRoots}(S, X_1, \text{maxDepth} - 1)$
 - 12: $(C_2, U_2) \leftarrow \text{IsolateRoots}(S, X_2, \text{maxDepth} - 1)$
 - 13: **return** $(C_1 \cup C_2, U_1 \cup U_2)$
-

Tactic

- Tactics can automate aspects of proof writing :
 - `simp` : simplifying expressions
 - `cases` : split the goal into cases
 - `aesop` : search for obvious proofs

Introduction

Interval
Arithmetic

Interval Newton
Method

Krawczyk
Method

Algorithm

Conclusion

Tactic

- Tactics can automate aspects of proof writing :
 - `simp` : simplifying expressions
 - `cases` : split the goal into cases
 - `aesop` : search for obvious proofs
- Our tactic `krawcheck` closes 3 kinds of goals

```
67
68 def V0 : Vecterval 2 := #v[dy[[0,1/2]], dy[[0,1]]]
69 def V1 : Vecterval 2 := #v[dy[[-1,0]], dy[[0,1]]]
70 def V2 : Vecterval 2 := #v[dy[[3/2, 2]], dy[[-5/2, 2]]]
71 def S : System 2 2 := poly[x1^3, x2^3, -3/2 * x1 * x2;
72   4 * x1^2 * x2, 9/4 * x1 * x2^2, -1/2 * x1, -5/2 * x2, 1]
73
74 example : V0.HasRoot S := by krawcheck
75 example : V1.HasNoRoot S := by krawcheck
76 example : V2.HasUniqueRoot S := by krawcheck
77
```

Conclusion

Contribution

- Formalizes the Krawczyk method, which uses transversality certify existence and uniqueness of roots
- Implements a branch-and-bound algorithm for isolating the roots in an interval.
- Introduces the `krawcheck` tactic to automate the root search and build the formal proof term.

Future Work

- Extend this framework to the case where $m < n$
- Implement Mean Value Form for multivariate polynomials
- Train an RL Agent to guess target groups for presentations, and verify with `krawcheck`